



MODELING TEST

Classical Optimization for Quantum Computing

Problem Definition

One important step of a quantum algorithm based on neutral-atom technology is the register design, in which the number of atoms and their position are provided in order to set up the quantum processing unit (QPU). As input, the algorithm receives some information about the instance of the problem to be solved (e.g., an instance of a combinatorial problem), such as a QUBO matrix Q , as the following example that represents an instance for the Set Partitioning problem:

$$Q = \begin{bmatrix} -17 & 10 & 10 & 10 & 0 & 20 \\ 10 & -18 & 10 & 10 & 10 & 20 \\ 10 & 10 & -29 & 10 & 20 & 20 \\ 10 & 10 & 10 & -19 & 10 & 10 \\ 0 & 10 & 20 & 10 & -17 & 10 \\ 20 & 20 & 20 & 10 & 10 & -28 \end{bmatrix}$$

Ideally, one places the atoms in such a way that the interaction energy between each pair of them will mimic the related coefficient from Q . This energy is based on the Van-der-Waals interactions and is calculated as following:

$$U_{ij} = \frac{C_6}{r_{ij}^6}$$

where C_6 is a constant given by the QPU, while r_{ij} is the Euclidian distance between atoms i and j . The final interaction matrix U given by a register is

$$U = \begin{bmatrix} U_{ii} & \cdots & U_{ij} \\ \vdots & \ddots & \vdots \\ U_{ji} & \cdots & U_{jj} \end{bmatrix}$$

It is worth mentioning that each QPU has its own technical constraints that must be respected. They are related to the maximum distance to the centre of the register that an atom can be put, and the minimal distance between any pair of atoms.

When solving a generic QUBO instance, the Register Design problem consists of choosing the positions of each atom so that the resulting pairwise interactions reproduce the entries of the QUBO matrix Q as accurately as possible, while satisfying all hardware constraints of the QPU.



Problem variations

Version 1: Free space

In this version of the problem, the atoms can be placed in a continuous 2-dimension space.

Version 2: Fixed Layouts

In this variant, atoms cannot be placed arbitrarily. Instead, one restricts the register design to a set of pre-calibrated layouts, each providing a predefined collection of trapping sites where atoms can be placed.

Modeling Questions

Please, choose at least one version of the problem and answer the following questions.

- 1 – Considering the decision version of the problem, provide a complexity reduction scheme using any classical graph/combinatorial problem.
- 2 – Provide a cost function to be optimized.
- 3 - Provide an upper-bound and a lower-bound for the cost function given any arbitrary instance of the problem.
- 4 - Provide a complete mathematical formulation that solves the chosen version of problem and optimizes the cost function proposed for question 2.
- 5 - Provide valid inequalities for the proposed formulation.
- 6 - Provide a heuristic to solve the chosen version of the problem.
- 7 - Provide a UML class diagram for encoding the problem and the proposed heuristic.
- 8 - Discuss the complexity of solving the proposed heuristic.
- 9 - Briefly discuss how you would apply the Genetic algorithm and the Tabu Search method to solve the problem. Which one do you think would be more appropriate to each version of the problem?

Coding Exercise

Please, provide a GitHub repository that hosts the implementation of the heuristic proposed in response to Question 6.